

DESENVOLVIMENTO DE UM WEBPROXY - SLSPROXY

Othavio Christmann Correa
Theo Corvello Soares

1 INTRODUÇÃO

Um WebProxy é um servidor que atua como intermediário entre o cliente e o servidor, servindo para bloquear, filtrar conteúdo e até mesmo mascarar o IP do cliente dentre outras funcionalidades. O WebProxy *SLSPProxy*¹ foi desenvolvido visando as funcionalidades citadas anteriormente. Possui implementação para filtragem de conteúdo HTML, bloqueio de HOST's e suporte ao HTTP Connect, método utilizado pelo HTTPS.

2 METODOLOGIA E ARQUITETURA

O *SLSPProxy* foi desenvolvido utilizando Sockets na linguagem Python. O programa inicia em uma porta definida no arquivo *.env*, para se conectar ao proxy, basta o cliente configurar, seja nas configurações do navegador ou do sistema operacional, o host e a porta utilizados. Dentro do programa, um objeto da classe Proxy é iniciado e, então, faz o bind para a porta e espera por conexões. Quando o cliente se conecta ao proxy, uma nova thread é iniciada para gerenciar os pedidos deste e o programa volta a escutar a porta, esperando um novo pedido de conexão. Dentro do handle, o proxy escuta um chamado do cliente (*Request Line + Header*), modifica a request line para obter o host, porta e alterar o URL completo para páginas locais (*www.youtube.com/video* → */video*). Após obter o host, verifica se o domínio ou algum subdomínio está presente no arquivo *blocked.json*, arquivo este que gerencia todos os domínios bloqueados pelo proxy. Caso presente, o proxy retorna uma página de erro 403 para o cliente e encerra sua conexão. Contudo, caso o domínio seja liberado, o proxy prossegue para a conexão com o host, respeitando o método chamado.



Figura 1 — Método HTTP Connect

¹ <https://github.com/TheoSoares/webproxy>

2.1 HTTP Connect

Com o chamado de um CONNECT, o proxy atua como uma ponte, ele abre uma conexão com o servidor, avisa o cliente que a conexão foi estabelecida (*HTTP/1.1 200 Connection Established*), e eles começam a trocar dados. O proxy recebe os dados enviados pelo cliente e repassa ao servidor ao mesmo tempo que recebe os dados do servidor e repassa para o cliente. Por conta do conteúdo criptografado, não é possível modificar os dados antes de repassar, para isto, seria necessário aplicar um outro método como MITM (Man in the Middle), sendo assim, o filtro de palavras funciona apenas para requests feitos nos outros métodos, como GET, POST etc por serem um chamado ao servidor e não uma conexão direta.

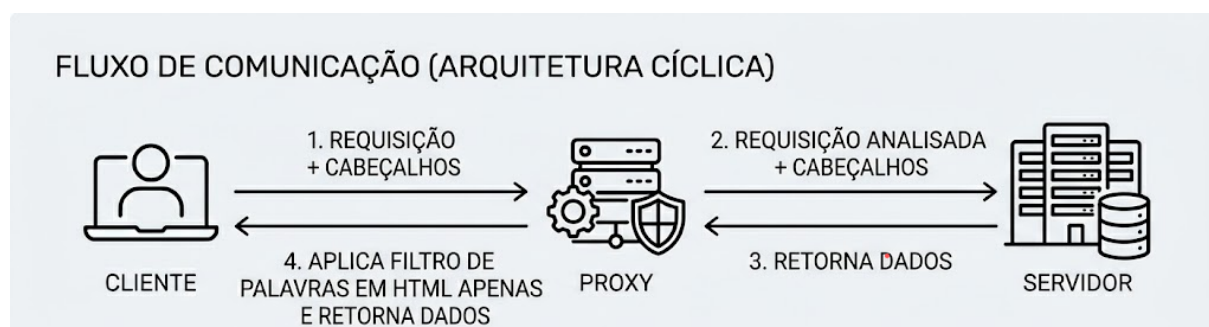


Figura 2 — Métodos HTTP convencionais

2.2 Outros Métodos HTTP

Para o restante dos métodos (GET, POST, PUT, DELETE, PATCH), o proxy recebe os dados do cliente e repassa ao servidor. Após repassar, começa a coletar e ler os dados enviados pelo servidor. De primeira mão, modifica o header para remover a chave Content-Length (com esta chave, o cliente só irá ler o body até onde o tamanho fizer jus ao valor, removendo isto, ele lê os dados recebidos até que a conexão seja fechada, e como o SLSPProxy não envia o HTML completo, mas sim por partes conforme recebe os dados do servidor, não é possível calcular o tamanho total do body, por isso deve ser removido) e modifica a chave Connection de Keep-Alive para closed quando possível (ao manter o Keep-Alive sem um Content-Length ou outro método de especificação do tamanho do body, o cliente fica carregando infinitamente a página, pois não reconhece quando finalizou o recebimento da requisição feita).

3 TECNOLOGIA

A tecnologia escolhida para o desenvolvimento deste WebProxy foi Socket TCP/IP, utilizando-os na linguagem Python. Esta tecnologia foi escolhida por conta de sua eficácia para esta área, visto que é a implementação na sua mais pura forma, recebendo dados por sockets, manipulando-os, conectando-se 'manualmente' aos servidores, repassando requisições etc... Além deste benefício de poder manipular de forma livre os dados enviados e recebidos, com esta tecnologia é possível implementar o método CONNECT de forma 'fácil', visto que, para não quebrar os dados criptografados o proxy deve apenas servir como uma ponte entre as conexões, algo que tem sua implementação mais dificultada em outras tecnologias comuns, como Flask.

4 DIFICULDADES

Como todo projeto desenvolvido do zero em uma área com pouca experiência, houveram dificuldades enfrentadas. Abaixo estão algumas junto a uma breve explicação do que é, como funciona e como foi resolvido:

1. Interpretação de Request: para iniciar o projeto, foi necessário entender como funcionava uma linha de requisição feita pelo navegador (cliente). A linha é composta por *MÉTODO HOST:PORTA/ROTA PROTOCOL Header[KEY: VALUE]*. Após conseguir fazer a divisão correta da request line, o host e a porta são utilizadas para abrir a conexão com o servidor, o método é o chamado da função do proxy (GET, POST etc... são tratados da mesma maneira, CONNECT possui uma função especial) e a requisição é alterada para remover o *HOST* e a *PORTA*, mantendo apenas a *ROTA*.

Exemplo de Request Line + Headers:

```
b'CONNECT static.meliuz.com.br:443 HTTP/1.1\r\nUser-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:151.0) Gecko/20100101 Firefox/151.0\r\nProxy-Connection: keep-alive\r\nConnection: keep-alive\r\nHost: static.meliuz.com.br:443\r\n\r\n'
```

2. Porta oculta: ocorreram erros ao tentar analisar a URL nos métodos HTTP, onde ela não era interpretada da maneira correta. Isto ocorria pois, no HTTP, a porta quando não especificada é padrão (80). Para o HTTPS, mesmo tendo 443 como porta padrão, ela sempre deve ser especificada.
3. Modificar cabeçalhos: o proxy estava repassando os dados corretamente para o cliente, mas ele às vezes cortava conteúdos, isto ocorria por conta do parâmetro *Content-Length* do cabeçalho que, como já explicado anteriormente, serve para dizer para o cliente quando ele deve parar de esperar mais conteúdo e renderizar a página. Para arrumar isto, foi necessário remover este parâmetro.
4. Parâmetro Connection: também na parte dos cabeçalhos, foi necessário alterar o parâmetro Connection, pois, com a remoção do parâmetro anterior, o cliente não sabia quando parar de esperar por dados novos. Após, ele passou a reconhecer o fim do arquivo como o encerramento da conexão. Isto pode atrapalhar o desempenho pois, caso o cliente queira solicitar um JavaScript ou CSS do mesmo host deve reabrir uma conexão e não reutilizar a antiga.
5. Método Connect: uma das maiores dificuldades foi entender o funcionamento do método connect, pois ele não é como os outros em que o host apenas repassa o seu conteúdo. Para iniciar a ponte, o que ocorre é que o cliente solicita conexão e então o proxy confirma para o cliente se a conexão com o host foi estabelecida ou não. Isto é, não adianta repassar a requisição CONNECT domínio PROTOCOLO diretamente para o servidor, pois o cliente fará tudo apenas após receber a confirmação de conexão por ponte e o servidor também estranhará a requisição e fechará a conexão por segurança.

5 APRENDIZADOS

A implementação deste proxy foi de grande aprendizado pois, ensinou como as requisições funcionam por baixo do pano, isto é, quais são os dados enviados, recebidos, o que significam alguns itens (request line, headers etc), qual a diferença entre HTTP e HTTPS além de como ambas funcionam.

6 USO DE IA

O modelo de IA utilizado foi o Gemini. Ele foi usado para esclarecimento de dúvidas e debug de erros, como por exemplo:

- 'O que significam os itens de uma requisição?': METHOD domain:port/route PROTOCOL Header[KEY: VALUE];
- 'Por que envio a request line para o https e o servidor e o cliente não conversam entre si?': A request line com CONNECT não deve ser repassada para o servidor, mas sim deve ser estabelecida uma conexão entre o proxy e o servidor e confirmar ao cliente que a conexão foi estabelecida, basta começar a enviar os dados para conversar com o servidor;
- 'Servidor não retorna dados corretos': a requisição estava sendo mantida original, então o servidor recebia seu próprio host, porta etc ao invés de receber apenas a rota local desejada;
- 'Request line sem porta especificada': para o método HTTP, a porta não é necessariamente especificada (quando assim, é usada a porta padrão 80) ao contrário do método HTTPS, que sempre é especificada na request line;
- 'Qual o código da Response que indica página bloqueada?': 403;
- 'Cliente está cortando a resposta': Content-Length não correspondia ao novo tamanho do HTML;
- 'Cliente renderizando página infinitamente': com a remoção do Content-Length, a página ficava sempre esperando mais dados, para isto, foi necessário fechar a conexão para ele entender como fim de recebimento.